Peer Application Evaluation

ARTI 503 Parallel Architecture and Programming, Term 1 2026-2027

Subgroup being evaluated: Group 1 – 9FS1

Subgroup completing the evaluation: Group 2 - 9FS1

Project summary

Ray tracing is a computer graphics technique that simulates how individual rays of light interact with physical surfaces to produce realistic 2D images of 3D scenes. Each scene contains objects, a virtual camera, and a light source. For every pixel, the camera sends a ray into the scene; the program finds the object the ray hits and calculates how light interacts with and reflects off the surface at that point to determine the pixel's final color. Because each pixel is sampled with multiple rays, and each ray needs to be checked against the objects several times in the scene, the process is computationally expensive and slow for large images on a single CPU core.

The group will first build a ray tracer in C++ that traces rays through a 3D scene and takes multiple samples per pixel to compute each pixel's color. Parallelization will then be implemented using OpenMP API, distributing the work across threads, each with its own random number generator to keep results independent. The implementation will be tested with different thread counts on local machines and in a cloud environment. The performance will be evaluated using execution time and speedup across different numbers of threads.

Relevance

This project is highly relevant to parallel architecture and programming. Ray tracing is an example of a data parallelism problem, where each pixel's color depends only on its own ray and the shared scene data, so pixels can be computed independently with no communication between them. This makes the problem a strong fit for multicore CPUs, and it is the same reason modern GPUs include dedicated ray-tracing hardware.

The project also addresses an important correctness issue in parallel programs. Ray tracing depends on random sampling, and a single random number generator shared between threads would either cause race conditions or require locking that slows the program down. Assigning each thread its own generator avoids both problems and shows awareness of how shared state affects parallel performance.

In addition, the proposal also shows awareness of the main challenges in parallel programming. The problem statement recognizes that some pixels take longer to compute than others, which causes load imbalance, and the plan to compare static, dynamic, and guided scheduling addresses this directly. Comparing critical, atomic, and reduction connects the project to the cost of synchronization covered in the course.

The experimental design aligns well with the course objectives. Varying thread counts and image sizes allows the group to measure speedup, efficiency, and scalability. Additionally, comparing both local machines and cloud resources will show how well the program scales on different hardware. Overall, the project is a practical and well-chosen way to apply core parallel computing concepts.

Clarity and writing

The application and idea are well organized, the problem is explained clearly, and the expected outcomes are specific and measurable. The proposal includes a table of contents and moves logically from the problem statement to objectives, project details, funding, and expected outcomes. However, there are several areas that need attention:

- The phrase "in addition to showing the below program" refers to a program that is not included in the document.
- The label "Cloud and processor comparison: Local and cloud comparison:" is repeated in the Expected Outcomes section.
- The Project Details section states that an EC2 instance "with a single processor and core" will be used, which contradicts the 16-vCPU c6i.4xlarge described in the Selected Funding Source section and the AWS form.

Feasibility

The core project is feasible. Sequential ray tracers in C++ are well documented, and OpenMP parallelization of the pixel loop is manageable. However, the proposal does not include a timeline, which makes it difficult to determine whether the work fits within the course period. The number of experiments is also large (several image sizes, three scheduling strategies, three synchronization methods, and two types of machines), so it should be planned carefully.

Suggested improvements

The project is strong and closely aligned with the course. The following changes would make the application clearer:

- Add a week-by-week timeline with milestones, plus a backup plan, such as running all experiments on laptops first, in case the AWS credits are delayed or rejected.
- Clarify how the AWS and laptop results will be compared, since the hardware specifications may differ significantly.
- Specify the experimental parameters: the scene or scenes used, image resolutions, samples per pixel, maximum ray depth, and the laptops' specifications.